L8: Unified Memory

Core idea

One pointer via `cudaMallocManaged(&p,N)` usable from CPU and GPU; runtime migrates data. Replaces both `malloc` and `cudaMalloc`. UM is for **programmability**, not speed; tuned manual `cudaMemcpy` usually wins.

Two regimes (memorize)

Pre-Pascal (Kepler/Maxwell): whole allocation migrates en masse at kernel launch; `cudaDeviceSynchronize()` brings it back. No oversubscription, no concurrent CPU/GPU access (CPU touch during kernel = **segfault**), no on-demand paging.

Pascal+: true demand paging. GPU touches non-resident page ⇒ page fault ⇒ driver migrates that page ⇒ access replays. Allows oversubscription (>GPU mem), concurrent CPU/GPU access, system-wide atomics. Many faults = slow.

Pattern

```
char *d; cudaMallocManaged(&d,N);
fread(d,1,N,fp);
qsort<<<...>>>(d,N,1,cmp);
cudaDeviceSynchronize(); // before CPU
    use
use_data(d); cudaFree(d);
```

Tuning APIs (Pascal+)

`cudaMemPrefetchAsync(p,len,dev,stream)` **moves** data ahead of time (`dev` may be `cudaCpuDeviceId`).

`cudaMemAdvise(p,n,hint,dev)` hints **never move data**:

- `SetReadMostly`: local read-only copy per processor; a write collapses all copies.
- `SetPreferredLocation`: suggests home; P2P map if possible, else migrate on fault.
- `SetAccessedBy`: direct P2P mapping for that processor, no faults, no movement.

`atomicAdd_system`: atomic across CPU + all GPUs (Pascal+; direct on NVLink, SW-assisted on PCIe).

UM shines for

Deep copies (struct w/ pointers), linked lists, C++ classes (overload new/delete), sparse/unpredictable access.

L9: MatMul & Tiling

Basic kernel

```
int Row=blockIdx.y*blockDim.y+threadIdx.
    y;
int Col=blockIdx.x*blockDim.x+threadIdx.
    x;
if(Row<W && Col<W){ float Pv=0;
  for(int k=0;k<W;++k)
    Pv+=M[Row*W+k]*N[k*W+Col];
  P[Row*W+Col]=Pv; }
```

2 global loads per 2 FLOPs: ratio 1, memory bound.

Tiling, 6 steps

1) identify shared tile 2) cooperative load to `__shared__` 3) `__syncthreads()` 4) consume from shared 5) `__syncthreads()` 6) next tile.

Tiled SGEMM

```
__shared__ float Mds[T][T], Nds[T][T];
int bx=blockIdx.x, by=blockIdx.y;
int tx=threadIdx.x, ty=threadIdx.y;
int Row=by*T+ty, Col=bx*T+tx;
float Pv=0;
for(int p=0;p<W/T;++p){
  Mds[ty][tx]=M[Row*W + p*T+tx];
  Nds[ty][tx]=N[(p*T+ty)*W + Col];
  __syncthreads();
  for(int k=0;k<T;++k)
    Pv+=Mds[ty][k]*Nds[k][tx];
  __syncthreads();
}
P[Row*W+Col]=Pv;
```

Tile size math (exam favorite)

T=16: 256 thr/blk; per phase 512 loads, 8192 mul/add ⇒ **16 FLOPs/load**. Shared = $2 \cdot 16^2 \cdot 4$B = **2KB** ⇒ 8 blocks in 16KB SM.

T=32: 1024 thr/blk; 2048 loads, 65536 ops ⇒ **32 FLOPs/load**. Shared = **8KB** ⇒ only 2 blocks/SM.

General: FLOPs per global load = T. Shared/blk = $2T^2 \cdot 4$ bytes.

Boundary tests (3 distinct!)

Load M: `Row<H && p*T+tx<W` else store 0.
Load N: `p*T+ty<H && Col<W` else store 0.
Write P: `Row<H && Col<W`.
Zeros make mul-add a no-op; threads with no valid P still help load tiles.

L10: Memory Coalescing

DRAM mechanics

Cell = capacitor + transistor; random access slow. **Burst:** one row read fills buffer, streamed in N transfers at interface speed (DDR2/3 core = 1/4 or 1/8 interface speed; buffer 8x interface width). **Banks** overlap to hide dead time. HBM2/3 = stacked dies + TSVs (A100 1555 GB/s, H100 3 TB/s).

The rule

Coalesced iff index = `A[(expr indep of threadIdx.x) + threadIdx.x]`; consecutive threads hit consecutive addresses in one burst section.

MatMul example

Row-major: `M[i][j]=M[i*W+j]`.
`N[k*W+Col]`, `Col=bx*T+tx`: consecutive in tx ⇒ **coalesced**.
`M[Row*W+k]`: address varies with ty not tx; column-walk patterns like `A[tid*W]` are **not** coalesced.
Tiling fixes coalescing: load tile coalesced into shared, then read shared freely (no coalescing rules in shared).

L11: Histogram / Atomics

Partitioning

Sectioned (contiguous chunk per thread): adjacent threads far apart ⇒ NOT coalesced. **Interleaved** (stride = total threads): adjacent threads adjacent memory ⇒ coalesced. Use interleaved on GPU.

Data race

`Old=Mem[x]; New=Old+1; Mem[x]=New` with 2 threads from 0 ends as **1 or 2**. Need atomic RMW: hardware serializes per address.

Atomics

`int atomicAdd(int*,int)` returns **OLD** value. Variants: Sub, Min, Max, Inc, Dec, Exch, CAS, And, Or, Xor. Scopes: `_system` (CPU+GPUs), default (device), `_block`.

Interleaved kernel

```
int i=threadIdx.x+blockIdx.x*blockDim.x;
int stride=blockDim.x*gridDim.x;
while(i<size){
  int pos=buf[i]-'a';
  if(pos>=0 && pos<26)
    atomicAdd(&histo[pos/4],1);
  i+=stride; }
```

Atomic cost

Each RMW = read latency + write latency, location locked meanwhile. Contended bin: throughput ≈ 1/latency ≪ bandwidth. **DRAM atomic ~1000+ cyc; L2 ~1/10 of that; shared mem very short, ~100x faster than DRAM.**

Privatization

Per-block private histogram in shared; merge once at end:

```
__shared__ unsigned histo_p[7];
if(threadIdx.x<7) histo_p[threadIdx.x
    ]=0;
__syncthreads();
while(i<size){
  atomicAdd(&histo_p[(buf[i]-'a')/4],1);
  i+=stride; }
__syncthreads();
if(threadIdx.x<7)
  atomicAdd(&histo[threadIdx.x],
            histo_p[threadIdx.x]);
```

Requires op **associative + commutative** and copy fits in shared. Too big? Partial privatization (hot bins only).

L12: Multi-GPU

Device management

CUDA calls target the *current* device; `cudaSetDevice(i)` switches. Switching while async work is in flight is fine, both GPUs run concurrently:

```
cudaSetDevice(0); k<<<...>>>();
cudaSetDevice(1); k<<<...>>>(); //0
    still runs
```

P2P

```
cudaDeviceCanAccessPeer(&can,dX,dY);
cudaDeviceEnablePeerAccess(peer,0);
cudaMemcpyPeerAsync(dst,dDev,src,sDev,n,
    s);
```

With P2P: shortest PCIe path, no host staging. Without: routed through host.

Numbers (PCIe gen2): P2P ~6.6 GB/s simplex, ~12 duplex (vs 5 via host); gen3 doubles. 4 GPUs via switches ~15 GB/s aggregate.

Halo exchange (1D decomp)

Stage 1: all send right, recv left. Stage 2: all send left, recv right. Concurrent copies, no link contention.

NUMA

Dual-IOH: remote D2H crosses QPI, extra hop. **Local ~6.3 GB/s vs remote ~4.3 GB/s.** Dual-IOH **blocks P2P across IOHs** (falls back to host staging). Pin threads near GPU: `numactl`, `GOMP_CPU_AFFINITY`.

L13: GPU Sharing, MPS

Problem + idea

Strong-scaling MPI: more ranks = less GPU work each; each process has its own context and the GPU **serializes contexts** (switch overhead, no overlap). **MPS** = one shared context via a server process; kernels/copies from different processes can overlap.

Requirements

Linux only; UVA; Tesla CC ≥ 3.5; CUDA 5.5+. Exclusive mode applies to the MPS *server*, not clients.

Hyper-Q / concurrency

Concurrent kernels since Fermi (CC 2.0) on different streams if resources allow: Fermi 16, Kepler 32. **Kepler Hyper-Q:** 32 HW work queues, one per stream, kills false inter-stream dependencies. Under MPS each client gets 2 queues per channel, 32 total. Streams/client ≤ channels: fine. Streams/client > channels: **false dependency, serialization**.

Setup (no code changes)

```
export CUDA_VISIBLE_DEVICES=0
nvidia-smi -i 0 -c EXCLUSIVE_PROCESS
```

```
nvidia-cuda-mps-control -d
```

Daemon spawns `nvidia-cuda-mps-server`; later processes route CUDA work through it. MPS does **not** spread work across GPUs (app handles affinity). Profile: `nvprof -profile-all-processes -o f_%p`.

L14: Dynamic Parallelism

Core idea

Kepler+ (CC 3.5+): a kernel can launch child kernels with normal `<<g,b>>` syntax. Unlocks: device-side library calls (cuBLAS), data-dependent parallelism (irregular bounds `for j=1..x[i]`), recursion, adaptive mesh refinement, batched nested parallelism (CPU freed).

Rules (memorize)

- **Launch is per-thread**: 2 threads at the launch line ⇒ 2 child grids.
- **Sync is per-block**: device `cudaDeviceSynchronize()` waits for launches by *any* thread in the block.
- `cudaDeviceSynchronize()` does **NOT** imply `__syncthreads()`; add it if you need a block barrier after.
- All device-side launches and copies are **async**; no sync launch.
- Streams/events are per-block, cannot pass to children. Constants set from host; textures/-surfaces host-bound; ECC errors at host.

Pattern

```
__global__ void dyn(float *d){
  int t=threadIdx.x;
  if(t%2) buf[t/2]=d[t]+d[t+1];
  __syncthreads();
  if(t==0){
    child<<<128,256>>>(buf);
    cudaDeviceSynchronize(); }
  __syncthreads(); // still required!
  cudaMemcpyAsync(d,buf,1024);
  cudaDeviceSynchronize();
}
```

Library call: one thread calls `cublasDgemm` + `cudaDeviceSynchronize()`, then `__syncthreads()` so all threads see the result.

Carryover from Midterm 1

Warp = **32 threads**. Warps/block = $\lceil$blockDim/32$\rceil$.
2D grid for $W \times H$ image, block $B_x \times B_y$: grid = $(\lceil W/B_x \rceil, \lceil H/B_y \rceil)$; ceiling div: `(n+b-1)/b`.
Divergent warps: boundary blocks where the `if(Row<H&&Col<W)` test splits a warp.
Coalescing check: is the index `base + threadIdx.x`?

Quick numbers & trivia

- atomic returns **old** value.
- `cudaMemAdvise` = hints, no movement; `cudaMemPrefetchAsync` = moves.
- Atomic latency: DRAM ≫ L2 (∼10x faster) ≫ shared (∼100x faster).
- Tiled SGEMM: FLOPs/load = T; shared/blk = $8T^2$ bytes (2 float tiles).
- P2P gen2: 6.6 simplex / 12 duplex GB/s; host path 5.
- NUMA: 6.3 local vs 4.3 remote GB/s D2H.
- Hyper-Q: 32 queues; MPS client: 2 per channel.
- DP needs CC 3.5+; MPS needs CC 3.5+, Linux, UVA.
- Pre-Pascal UM: CPU touch during kernel segfaults; Pascal+ page-faults.

Worked example: launch math

1000×1000 image, 16×16 blocks:
Grid = $\lceil 1000/16 \rceil^2 = 63 \times 63 = 3969$ blocks.
Threads = 3969·256 = 1,016,064 launched (16,064 idle past bounds).
Warps/block = 256/32 = 8; total warps = 31,752.
Divergence: a 16×16 block has warps spanning 2 rows of 16. Edge blocks (right col, bottom row) have warps where `if(Row<H&&Col<W)` splits inside the warp ⇒ divergent; interior blocks have none.

Tiled occupancy check

T=32, 48KB shared/SM, 2048 thr/SM max: shared allows 48/8 = 6 blocks but thread limit allows 2048/1024 = 2 ⇒ **2 blocks/SM** (thread bound). Always take the min over all limits (threads, shared, registers, blocks).

API quick reference

```
kernel<<<grid,block,shmemB,stream
    >>>(...);
dim3 g(gx,gy), b(bx,by);
cudaMalloc(&p,N); cudaFree(p);
cudaMemcpy(dst,src,N,
    cudaMemcpyHostToDevice);
cudaMemcpyAsync(d,s,N,kind,stream);
cudaStreamCreate(&s); cudaStreamDestroy(
    s);
cudaMallocManaged(&p,N);
cudaMemPrefetchAsync(p,N,dev,stream);
cudaMemAdvise(p,N,hint,dev);
cudaSetDevice(i); cudaGetDeviceCount(&n)
    ;
cudaDeviceCanAccessPeer(&ok,a,b);
cudaDeviceEnablePeerAccess(peer,0);
cudaMemcpyPeerAsync(d,dDev,s,sDev,N,st);
cudaDeviceSynchronize();
old = atomicAdd(&x,v);
```

Which optimization when

- Redundant global reads across threads ⇒ **tiling** in shared memory.
- Strided / scattered warp access ⇒ restructure for **coalescing** (or stage through shared).
- Many threads update few locations ⇒ **atomics** + **privatization**.
- Data too big for one GPU or need more FLOPs ⇒ **multi-GPU** + **P2P** halo exchange.
- Many small processes underusing GPU ⇒ **MPS**.
- Work amount unknown until runtime ⇒ **dynamic parallelism**.
- Complex pointer structures, prototyping ⇒ **unified memory** (+prefetch/advise to claw back perf).